

AssemblyAI Integration Investigation

Executive Summary

AssemblyAI was **never fully integrated** — it was part of a legacy `AudioClarification` system (v129) that was **replaced** by `ClarifyAudioPanel` during a major refactor on May 19, 2026. The API key is valid, speaker diarization works perfectly in testing, and re-enabling it would give dramatically better speaker detection. **Recommended: build it as an optional “Advanced Speaker Detection” feature.**

1. What Was Found

Files Referencing AssemblyAI

File	Role	Status
<code>.env.local</code>	API key: <code>b38fca044343485db6-ab665c8af1b172</code>	✅ Key is VALID (tested)
<code>app/hooks/useChunkedTranscription.ts</code>	Hook that calls <code>/api/assemblyai-transcription</code>	⚠️ Code exists, but API route doesn't
<code>app/hooks/useAudioClarification.tsx</code>	v129 <code>AudioClarification</code> component (2,550 lines)	⚠️ Legacy, not used by watch page
<code>app/lib/speakerDiarization.ts</code>	Python script interface for speaker detection	❌ Python script doesn't exist
<code>app/lib/voiceAssignment.ts</code>	Speaker → voice mapping logic	✅ Functional but unused
<code>lib/speakerDiarization.ts</code>	Duplicate of above	❌ Same missing script
<code>lib/voiceAssignment.ts</code>	Duplicate of above	✅ Functional but unused

Missing Pieces (Never Created)

- `/api/assemblyai-transcription` **route** — The API endpoint that `useChunkedTranscription` tries to call does NOT exist
- `python_services/speaker_diarization.py` — The Python script that `speakerDiarization.ts` tries to execute does NOT exist
- assemblyai npm package** — Not in `package.json`

2. Why It Was Abandoned

Root Cause: Architecture Replacement

The git history tells the story clearly:

Date	Commit	What Happened
May 19, 18:41	41c3405	“Restore translation/audio clarification files” — The legacy system (<code>useAudioClarification v129</code> + <code>useChunkedTranscription</code> + Python diarization) was bulk-restored from backup as Phase 1 of a restoration plan
May 19, 20:13	fcb5752	“Integrate ClarifyAudioPanel into watch page as section 7” — ClarifyAudioPanel replaced AudioClarification as the active component. The legacy code stayed in the repo but was never wired up
May 19–26	20+ commits	All subsequent work focused on <code>ClarifyAudioPanel</code> — gap-based detection, voice assignment, frozen maps, etc.

The Legacy System Was Never Working

The `useChunkedTranscription` hook calls `/api/assemblyai-transcription` — but that route was **never created**. The hook was restored from backup along with 10,000+ lines of code, but the backend it depended on was missing. The restoration commit message even documents this:

“Missing items documented in RESTORATION_PLAN.md: API routes: `/api/multi-voice-tts`, `/api/process-video`”

The multi-voice-tts and process-video routes WERE later created, but the AssemblyAI transcription route was NOT — it was simply skipped in favor of the YouTube caption approach used by `ClarifyAudioPanel`.

It Wasn’t “Abandoned” — It Was Never Finished

The legacy code represents a **planned but incomplete** AssemblyAI integration. The hooks and type definitions were written, but the server-side infrastructure (API route, audio extraction, Python diarization) was never built. When `ClarifyAudioPanel` was created as a simpler alternative using free YouTube captions, the AssemblyAI path was deprioritized.

3. API Key Validation Test

Tested on May 26, 2026:

```
POST https://api.assemblyai.com/v2/transcript
  audio_url: "https://assembly.ai/wildfires.mp3"
  speaker_labels: true
  speech_models: ["universal-2"]

Result:  SUCCESS
Transcript ID: 5252a0e0-2628-4585-a775-78652d200ef0
Status: completed
Speakers found: 2 (Speaker A, Speaker B)
Utterances: 20

Speaker A: "Smoke from hundreds of wildfires in Canada..."
Speaker B: "Good morning."
Speaker A: "So what is it about the conditions..."
Speaker B: "Well, there's a couple of things..."
```

The API key works. Speaker diarization works. The output quality is excellent.

API Changes Since Legacy Code Was Written

The legacy `useChunkedTranscription` code would need minor updates:

- `speech_model` parameter → now `speech_models` (array, e.g., `["universal-2"]`)
- Response format is unchanged: `utterances[]` with `speaker`, `start`, `end`, `text`
- Speaker labels still use letter format: `"A"`, `"B"`, `"C"` (not numbered)

4. Current Pricing (May 2026)

Component	Cost	For 15-min video
Base transcription (Universal-2)	\$0.15/hour	\$0.0375
Speaker diarization add-on	\$0.02/hour	\$0.005
Total per video	\$0.17/hour	\$0.0425

That's about 4 cents per 15-minute video. Much cheaper than previously estimated (\$0.23).

New accounts get **\$50 in free credits** — that's ~294 hours of transcription + diarization, or about **1,176 fifteen-minute videos**.

5. Comparison: Current vs AssemblyAI

Current Approach (Gap-Based Detection)

YouTube Captions → Gap Analysis → Guess Speaker Boundaries → Voice Assignment

Aspect	Rating	Notes
Accuracy	✗ Poor	Guesses speakers from pause length — can't tell WHO is speaking
Cost	✓ Free	Uses YouTube captions already fetched
Speed	✓ Instant	Client-side, no API calls
Conversations	✗ Fails	YouTube captions overlap, all gaps negative
Monologues	⚠ OK	Single speaker detected correctly
Dependencies	✓ None	Pure client-side JavaScript
Maintenance	⚠ High	Constant tweaking of thresholds, fallbacks

AssemblyAI Approach

YouTube Audio → AssemblyAI API → ML Speaker Detection → Voice Assignment

Aspect	Rating	Notes
Accuracy	✓ Excellent	ML-based from actual audio waveforms
Cost	✓ Very Cheap	~\$0.04 per 15-min video
Speed	⚠ 30-60 seconds	Needs to process audio
Conversations	✓ Great	Built for multi-speaker content
Monologues	✓ Great	Correctly detects single speaker
Dependencies	⚠ API	Needs internet + valid API key
Maintenance	✓ Low	AssemblyAI handles the ML complexity

The Gap That Matters

For **Easy German videos** (Larry's main use case) — conversational videos with 2-3 speakers — gap-based detection is fundamentally wrong because:

1. YouTube captions are timestamped by **phrase timing**, not by **who's speaking**
2. A 3-second pause might be the same person pausing, not a speaker change
3. A speaker change mid-sentence has NO gap at all
4. Overlapping captions (negative gaps) make most detection methods fail

AssemblyAI analyzes the **actual audio** — it hears voice characteristics, not just timing.

6. Hybrid Approach (Recommended)

Instead of replacing the current system, **add AssemblyAI as an optional upgrade**:

Default Path (Free, Instant):

YouTube Captions → Gap Detection → Voice Assignment

Advanced Path (Optional, ~\$0.04/video, 30-60s **wait**):

YouTube Audio URL → AssemblyAI → Speaker Labels →
Match to YouTube Caption Timestamps → Voice Assignment

How the Hybrid Would Work

1. **User loads video** → Normal YouTube caption path fires immediately (free, instant)
2. **UI shows:** “🎤 3 speakers detected (estimated). Want better detection? [Use Advanced Detection]”

3. If user clicks Advanced:

- Extract audio URL via existing `/api/video-stream` route (uses yt-dlp, already built!)
- Submit to AssemblyAI with `speaker_labels: true`
- Poll for result (30-60 seconds, show progress bar)
- Match AssemblyAI speaker labels to YouTube caption timestamps
- Replace gap-based speakers with ML-detected speakers

4. Voice assignment proceeds as normal — just with correct speaker labels now

Why This Works

- `/api/video-stream` already exists and extracts audio URLs from YouTube
- AssemblyAI returns timestamps that can be aligned with YouTube captions
- The existing `frozenVoiceMapRef` system handles voice assignment regardless of detection method
- No changes needed to the TTS pipeline — only the speaker labels change

7. Implementation Plan

Phase 1: Basic AssemblyAI Integration (4-6 hours)

New files:

app/**api**/assemblyai-diarize/route.ts ☐ API route to submit + poll

Modified files:

app/components/ClarifyAudioPanel.tsx ☐ Add "Use Advanced Detection" button
 app/watch/page.tsx ☐ Wire up the button
package.json ☐ Add assemblyai **package** (optional, can
 use raw fetch)

API Route (`/api/assemblyai-diarize/route.ts`)

```
// POST: Submit audio for diarization
// - Get audio URL from /api/video-stream
// - Submit to AssemblyAI with speaker_labels: true
// - Return transcript ID

// GET: Poll for result
// - Check status by transcript ID
// - When completed, return utterances with speaker labels
```

Speaker Label Mapping

```
// AssemblyAI returns: { speaker: "A", start: 14000, end: 26000, text: "..." }
// YouTube captions:   { start: 14.2, end: 17.5, text: "..." }
//
// Match by timestamp overlap:
// For each YouTube caption segment, find the AssemblyAI utterance
// that overlaps the most → assign that speaker label
```

Phase 2: UI Integration (2-3 hours)

- Add "🧠 Advanced Speaker Detection" button to Speaker Voices panel
- Show progress: "Analyzing audio... (30s)" with spinner

- When done: update speaker labels, re-run voice assignment
- Persist AssemblyAI results in sessionStorage (don't re-process same video)

Phase 3: Polish (1-2 hours)

- Error handling (API failures, timeout, invalid audio URL)
- Cost tracking (show user how many free credits remain)
- Caching (don't re-analyze same video within same session)

Total estimated effort: 7-11 hours

8. Recommendation

Short-term: Keep the current fixes

The recent commits (top-N gaps, speaker override, frozen map fixes) make the gap-based system **functional enough** for demo purposes. It assigns 3 distinct voices that rotate through the transcript.

Medium-term: Build the hybrid approach

This is the highest-impact improvement possible for Tutorial Clarity's audio clarification feature. For ~\$0.04 per video and 7-11 hours of dev work:

- Speaker detection goes from "educated guess" to "ML-accurate"
- Conversational videos (Easy German, interviews) get correct speaker assignment
- The existing voice assignment pipeline (frozen maps, radio buttons, TTS) works unchanged
- It's optional — free users keep the instant gap-based detection

What NOT to do

- **Don't re-enable the legacy `AudioClarification v129` component** — it's 2,550 lines of complex code with its own audio pipeline that conflicts with `ClarifyAudioPanel`. It would be a massive regression.
- **Don't install the `assemblyai` npm package** — it's unnecessary overhead. Raw `fetch()` to their REST API is simpler and already proven in testing.
- **Don't make AssemblyAI required** — keep it optional so the app works without an API key or internet connection.

Appendix: File Map

Active System (`ClarifyAudioPanel`)

<code>app/watch/page.tsx</code>	← Watch page, Speaker Voices UI
<code>app/components/ClarifyAudioPanel.tsx</code>	← Audio panel, <code>detectSpeakers()</code> , <code>generateSeg()</code>
<code>app/api/multi-voice-tts/route.ts</code>	← TTS generation
<code>app/api/process-video/route.ts</code>	← Transcript + translation
<code>app/api/transcript/route.ts</code>	← YouTube transcript fetching

Legacy System (AudioClarification — NOT active)

app/hooks/useAudioClarification.tsx	← v129, 2,550 lines, NOT used
app/hooks/useChunkedTranscription.ts	← AssemblyAI hook, route missing
app/hooks/useAudioTranslation.ts	← 4,232 lines, NOT used
app/hooks/useClarifyAudio.ts	← Engine wrapper, NOT used
app/lib/speakerDiarization.ts	← Python interface, script missing
app/lib/voiceAssignment.ts	← Voice mapping, NOT used
app/lib/clientSpeakerDetection.ts	← v42 client detection, NOT used

Infrastructure (Useful for AssemblyAI Integration)

app/api/video-stream/route.ts	Extracts audio URLs via yt-dlp KEY FOR AS-
SEMBLYAI	
.env.local	Contains valid ASSEMBLYAI_API_KEY

Investigation completed: May 26, 2026
Branch: feature/v153-truly-clean
Investigator: Abacus AI Agent